

Trabalho Final — Benchmark de Otimização Numérica

Aluno: Renyer Montefusco Levy

Matrícula: 2242917

Objetivo

Resolver e comparar problemas de benchmark de otimização não linear utilizando Julia e solvers de otimização. O trabalho está dividido em duas partes:

1. **AMPL/NLP Benchmark** — 47 problemas em formato AMPL.
2. **COCONUT/GAMS Benchmark** — problemas em formato GAMS com solução de referência disponível.

Os solvers considerados são:

- Ipopt
- MadNLP
- NLOpt
- Optim
- Uno

Devido às limitações computacionais do equipamento utilizado, todas as execuções são limitadas a **100 iterações** ou ao parâmetro equivalente disponível em cada solver.

Critério experimental

As execuções registram:

- fonte do problema;
- formato original;
- nome do problema;
- solver utilizado;
- status retornado;
- valor objetivo obtido;
- valor objetivo de referência, quando disponível;
- gap de otimalidade;
- tempo de execução;
- número de iterações, quando disponível;
- mensagem técnica retornada pela execução.

O gap de otimalidade é calculado por:

$$\text{gap} = \frac{|f_{\text{obtido}} - f_{\text{referência}}|}{\max(1, |f_{\text{referência}}|)}$$

1. Configuração dos caminhos

```
In [1]: # Esta célula define os caminhos principais do experimento.
# Deve ser executada antes das demais células.

MAX_ITER = 100

function detectar_ampl()
    candidatos = String[]

    caminho_path = Sys.which("ampl")
    if caminho_path != nothing
        push!(candidatos, String(caminho_path))
    end

    push!(candidatos, "C:/AMPL/ampl.exe")
    push!(candidatos, raw"C:\AMPL\ampl.exe")

    for c in candidatos
        if isfile(c)
            return c
        end
    end

    # Última tentativa: usar o comando pelo PATH do sistema.
    return "ampl"
end

AMPL_EXE = detectar_ampl()
AMPL_SOURCE_DIR = raw"C:\AMPL_NLP_47\source"
AMPL_WORK_DIR = raw"C:\AMPL_NLP_47\execucao"
AMPL_RUN_DIR = joinpath(AMPL_WORK_DIR, "run")
AMPL_LOG_DIR = joinpath(AMPL_WORK_DIR, "logs")
RESULTS_DIR = "results"

mkpath(AMPL_WORK_DIR)
mkpath(AMPL_RUN_DIR)
mkpath(AMPL_LOG_DIR)
mkpath(RESULTS_DIR)

AMPL_DISPONIVEL = isfile(AMPL_EXE) || Sys.which("ampl") != nothing

println("Limite de iterações: ", MAX_ITER)
println("AMPL: ", AMPL_EXE)
println("Pasta AMPL/NLP: ", AMPL_SOURCE_DIR)
println("Pasta de resultados: ", RESULTS_DIR)
println("AMPL disponível? ", AMPL_DISPONIVEL)
println("Pasta dos modelos AMPL encontrada? ", isdir(AMPL_SOURCE_DIR))
```

```
Limite de iterações: 100
AMPL: C:\Users\Renyer Levy\AMPL\ampl.exe
Pasta AMPL/NLP: C:\AMPL_NLP_47\source
Pasta de resultados: results
AMPL disponível? true
Pasta dos modelos AMPL encontrada? true
```

2. Pacotes Julia

```
In [2]: import Pkg
```

```

# Pacotes externos necessários para a execução principal.
pacotes_externos = [
    "DataFrames",
    "CSV",
    "JuMP",
    "Ipopt"
]

for pacote in pacotes_externos
    try
        @eval using $(Symbol(pacote))
        println("OK: ", pacote)
    catch erro
        println("Instalando pacote: ", pacote)
        Pkg.add(pacote)
        @eval using $(Symbol(pacote))
        println("OK: ", pacote)
    end
end

# Bibliotecas padrão do Julia.
using Dates
using Statistics
using Printf

println("Pacotes principais carregados.")
println("DataFrame definido? ", isdefined(Main, :DataFrame))
println("Model definido? ", isdefined(Main, :Model))

```

Instalando pacote: DataFrames

```

    Updating registry at `C:\Users\Renyer Levy\.julia\registries\General.toml`
    Resolving package versions...
    Installed DataStructures ─── v0.19.5
    Installed OrderedCollections – v1.8.2
    Updating `C:\Users\Renyer Levy\.julia\environments\v1.12\Project.toml`
[a93c6f00] + DataFrames v1.8.2
    Updating `C:\Users\Renyer Levy\.julia\environments\v1.12\Manifest.toml`
[34da2185] + Compat v4.18.1
[a8cc5b0e] + Crayons v4.1.1
[9a962f9c] + DataAPI v1.16.0
[a93c6f00] + DataFrames v1.8.2
[864edb3b] + DataStructures v0.19.5
[e2d170a0] + DataValueInterfaces v1.0.0
[842dd82b] + InlineStrings v1.4.5
[41ab1584] + InvertedIndices v1.3.1
[82899510] + IteratorInterfaceExtensions v1.0.0
[b964fa9f] + LaTeXStrings v1.4.0
[e1d29d7a] + Missings v1.2.0
⚠ [bac558e1] + OrderedCollections v1.8.2
[2dfb63ee] + PooledArrays v1.4.3
[08abe8d2] + PrettyTables v3.3.2
[189a3867] + Reexport v1.2.2
[91c51154] + SentinelArrays v1.4.10
[a2af1166] + SortingAlgorithms v1.2.2
[10745b16] + Statistics v1.11.1
[892a3eda] + StringManipulation v0.4.4
[3783bdb8] + TableTraits v1.0.1
[bd369af6] + Tables v1.12.1
[9fa8497b] + Future v1.11.0
    Info Packages marked with ⚠ have new versions available but compatibility co
nstraints restrict them from upgrading. To see why use `status --outdated -m`
    Precompiling packages...
      3017.0 ms ✓ InlineStrings → ParsersExt
      7801.8 ms ✓ OrderedCollections
      5972.9 ms ✓ DataStructures
      2284.9 ms ✓ SortingAlgorithms
      8291.2 ms ✓ Tables
      2767.1 ms ✓ StructUtils → StructUtilsTablesExt
      97273.4 ms ✓ PrettyTables
     163684.9 ms ✓ DataFrames
      8 dependencies successfully precompiled in 288 seconds. 83 already precompiled.

```

OK: DataFrames

Instalando pacote: CSV

```

    Resolving package versions...
    Updating `C:\Users\Renyer Levy\.julia\environments\v1.12\Project.toml`
[336ed68f] + CSV v0.10.16
    Updating `C:\Users\Renyer Levy\.julia\environments\v1.12\Manifest.toml`
[336ed68f] + CSV v0.10.16
[944b1d66] + CodecZlib v0.7.8
[48062228] + FilePathsBase v0.9.24
[3bb67fe8] + TranscodingStreams v0.11.3
[ea10d353] + WeakRefStrings v1.4.3
[76ecee3] + WorkerUtilities v1.6.1
[a63ad114] + Mmap v1.11.0
    Precompiling packages...
      3356.1 ms ✓ WeakRefStrings
      76827.1 ms ✓ CSV
      2 dependencies successfully precompiled in 84 seconds. 97 already precompiled.

```

OK: CSV

Instalando pacote: JuMP

```

Resolving package versions...
Installed ForwardDiff — v1.4.1
Installed NaNMath — v1.1.4
Installed DiffRules — v1.16.0
Installed LogExpFunctions — v1.0.1
Installed SpecialFunctions — v2.8.0
Installed MathOptInterface — v1.51.1
Updating `C:\Users\Renyer Levy\.julia\environments\v1.12\Project.toml`
[4076af6c] + JuMP v1.30.1
Updating `C:\Users\Renyer Levy\.julia\environments\v1.12\Manifest.toml`
[523fee87] + CodecBzip2 v0.8.5
[bbf7d656] + CommonSubexpressions v0.3.1
[163ba53b] + DiffResults v1.1.0
[b552c78f] + DiffRules v1.16.0
[ffbed154] + DocStringExtensions v0.9.5
[f6369f11] + ForwardDiff v1.4.1
[92d709cd] + IrrationalConstants v0.2.6
[4076af6c] + JuMP v1.30.1
[2ab3a3ac] + LogExpFunctions v1.0.1
[1914dd2f] + MacroTools v0.5.16
[b8f27783] + MathOptInterface v1.51.1
[d8a4904e] + MutableArithmetics v1.8.0
[77ba4419] + NaNMath v1.1.4
[276daf66] + SpecialFunctions v2.8.0
[1e83bf80] + StaticArraysCore v1.4.4
[6e34b625] + Bzip2_jll v1.0.9+0
[efe28fd5] + OpenSpecFun_jll v0.5.6+0
[8dfed614] + Test v1.11.0
[05823500] + OpenLibm_jll v0.8.7+0
Precompiling packages...
 4394.3 ms ✓ LogExpFunctions
 8366.0 ms ✓ NaNMath
 8250.6 ms ✓ SpecialFunctions
 1767.3 ms ✓ DiffRules
18274.1 ms ✓ ForwardDiff
137216.1 ms ✓ MathOptInterface
126341.4 ms ✓ Ipopt → IpoptMathOptInterfaceExt
159601.2 ms ✓ JuMP
 8 dependencies successfully precompiled in 337 seconds. 113 already precompiled.

```

OK: JuMP

Instalando pacote: Ipopt

```

Resolving package versions...
Updating `C:\Users\Renyer Levy\.julia\environments\v1.12\Project.toml`
[b6b21f68] + Ipopt v1.14.3
Manifest No packages added to or removed from `C:\Users\Renyer Levy\.julia\environments\v1.12\Manifest.toml`

```

OK: Ipopt

Pacotes principais carregados.

DataFrame definido? true

Model definido? true

3. Estrutura da tabela de resultados

```

In [3]: if !isdefined(Main, :DataFrame)
          error("Execute primeiro a célula 2 – Pacotes Julia. Ela deve terminar com: DataFrame")
        end

resultados = DataFrame(
    parte = String[],
    fonte = String[],
    formato = String[],
    problema = String[],

```

```

solver = String[],
status = String[],
objetivo_obtido = Float64[],
objetivo_referencia = Float64[],
gap = Float64[],
tempo_segundos = Float64[],
iteracoes = Union{Missing, Int}[],
mensagem = String[]
)

function calcular_gap(objetivo_obtido, objetivo_referencia)
    if isnan(objetivo_obtido) || isnan(objetivo_referencia)
        return NaN
    end
    return abs(objetivo_obtido - objetivo_referencia) / max(1.0, abs(objetivo_referencia))
end

function texto_curto(txt; limite = 300)
    s = replace(string(txt), '\n' => ' ')
    return length(s) <= limite ? s : s[1:limite] * "..."
end

function registrar_resultado!(;
    parte,
    fonte,
    formato,
    problema,
    solver,
    status,
    objetivo_obtido = NaN,
    objetivo_referencia = NaN,
    tempo_segundos = NaN,
    iteracoes = missing,
    mensagem = ""
)
    gap = calcular_gap(objetivo_obtido, objetivo_referencia)

    push!(
        resultados,
        (
            parte,
            fonte,
            formato,
            problema,
            solver,
            string(status),
            objetivo_obtido,
            objetivo_referencia,
            gap,
            tempo_segundos,
            iteracoes,
            texto_curto(mensagem)
        )
    )

    return resultados
end

first(resultados, 0)

```

Out[3]: 0×12 DataFrame

Row	parte	fonte	formato	problema	solver	status	objetivo_obtido	objetivo_referencia	gap
	String	String	String	String	String	String	Float64	Float64	Float
◀	◀								
▶	▶								

Parte 1 — AMPL/NLP Benchmark

A primeira parte utiliza os 47 problemas do benchmark AMPL/NLP. Os modelos são lidos no formato `.mod` e executados pelo AMPL a partir do Julia. Cada solver é testado quando estiver disponível no ambiente.

4. Lista dos 47 problemas AMPL/NLP

```
In [4]: problemas_ampl = [  
    "arki0003",  
    "arki0009",  
    "bearing_400",  
    "camshape_6400",  
    "c1n1beam",  
    "cont5_1_1",  
    "cont5_2_1_1",  
    "cont5_2_2_1",  
    "cont5_2_3_1",  
    "cont5_2_4_1",  
    "corkscrw",  
    "dirichlet120",  
    "dtoc1nd",  
    "dtoc2",  
    "elec_400",  
    "ex1_160",  
    "ex1_320",  
    "ex4_2_160",  
    "ex4_2_320",  
    "ex8_2_2",  
    "ex8_2_3",  
    "gasoil_3200",  
    "henon120",  
    "lane_emden120",  
    "marine_1600",  
    "NARX_CFy",  
    "nq1180",  
    "optmass",  
    "pinene_3200",  
    "qcqp500-3c",  
    "qcqp500-3nc",  
    "qcqp750-2c",  
    "qcqp750-2nc",  
    "qcqp1000-1nc",  
    "qcqp1000-2c",  
    "qcqp1000-2nc",  
    "qcqp1500-1c",  
    "qcqp1500-1nc",  
    "qssp180",  
    "robot_1600",  
    "robot_a",  
    "robot_b",  
]
```

```

    "robot_c",
    "rocket_12800",
    "steering_12800",
    "svanberg",
    "WM_CFy"
]

arquivos_ampl = DataFrame(
    problema = problemas_ampl,
    arquivo = [joinpath(AMPL_SOURCE_DIR, p * ".mod") for p in problemas_ampl],
    existe = [isfile(joinpath(AMPL_SOURCE_DIR, p * ".mod")) for p in problemas_ampl]
)

display(arquivos_ampl)
println("Total esperado: ", length(problemas_ampl))
println("Arquivos encontrados: ", count(arquivos_ampl.existe))

if count(arquivos_ampl.existe) != length(problemas_ampl)
    println("Atenção: existem arquivos .mod ausentes. Conferir a pasta AMPL_SOURCE_D:
end

```


47×3 DataFrame

22 rows omitted

Row	problema	arquivo	existe
	String	String	Bool
1	arki0003	C:\\AMPL_NLP_47\\source\\arki0003.mod	true
2	arki0009	C:\\AMPL_NLP_47\\source\\arki0009.mod	true
3	bearing_400	C:\\AMPL_NLP_47\\source\\bearing_400.mod	true
4	camshape_6400	C:\\AMPL_NLP_47\\source\\camshape_6400.mod	true
5	clnlbeam	C:\\AMPL_NLP_47\\source\\clnlbeam.mod	true
6	cont5_1_l	C:\\AMPL_NLP_47\\source\\cont5_1_l.mod	true
7	cont5_2_1_l	C:\\AMPL_NLP_47\\source\\cont5_2_1_l.mod	true
8	cont5_2_2_l	C:\\AMPL_NLP_47\\source\\cont5_2_2_l.mod	true
9	cont5_2_3_l	C:\\AMPL_NLP_47\\source\\cont5_2_3_l.mod	true
10	cont5_2_4_l	C:\\AMPL_NLP_47\\source\\cont5_2_4_l.mod	true
11	corkscrw	C:\\AMPL_NLP_47\\source\\corkscrw.mod	true
12	dirichlet120	C:\\AMPL_NLP_47\\source\\dirichlet120.mod	true
13	dtoc1nd	C:\\AMPL_NLP_47\\source\\dtoc1nd.mod	true
⋮	⋮	⋮	⋮
36	qcqp1000-2nc	C:\\AMPL_NLP_47\\source\\qcqp1000-2nc.mod	true
37	qcqp1500-1c	C:\\AMPL_NLP_47\\source\\qcqp1500-1c.mod	true
38	qcqp1500-1nc	C:\\AMPL_NLP_47\\source\\qcqp1500-1nc.mod	true
39	qssp180	C:\\AMPL_NLP_47\\source\\qssp180.mod	true
40	robot_1600	C:\\AMPL_NLP_47\\source\\robot_1600.mod	true
41	robot_a	C:\\AMPL_NLP_47\\source\\robot_a.mod	true
42	robot_b	C:\\AMPL_NLP_47\\source\\robot_b.mod	true
43	robot_c	C:\\AMPL_NLP_47\\source\\robot_c.mod	true
44	rocket_12800	C:\\AMPL_NLP_47\\source\\rocket_12800.mod	true
45	steering_12800	C:\\AMPL_NLP_47\\source\\steering_12800.mod	true
46	svanberg	C:\\AMPL_NLP_47\\source\\svanberg.mod	true
47	WM_CFy	C:\\AMPL_NLP_47\\source\\WM_CFy.mod	true



Total esperado: 47

Arquivos encontrados: 47

5. Solvers previstos para a Parte AMPL/NLP

A execução no formato AMPL é realizada por chamada externa ao AMPL a partir do Julia. O notebook tenta os solvers listados e registra falhas quando algum deles não estiver disponível no ambiente.

```
In [5]: solvers_ampl = ["ipopt", "madnlp", "nlopt", "optim", "uno"]
        solver_options_ampl = Dict{
```

```

"ipopt" => ("ipopt_options", "max_iter=100 print_level=0"),
"madnlp" => ("madnlp_options", "max_iter=100"),
"nlopt" => ("nlopt_options", "maxeval=100"),
"optim" => ("optim_options", "iterations=100"),
"uno" => ("uno_options", "max_iter=100")
)

DataFrame(
  solver = solvers_ampl,
  limite_iteracoes = fill(MAX_ITER, length(solvers_ampl))
)

```

Out[5]: 5×2 DataFrame

	solver	limite_iteracoes
	String	Int32
1	ipopt	100
2	madnlp	100
3	nlopt	100
4	optim	100
5	uno	100

6. Funções de execução AMPL/NLP

```

In [6]: function ampl_path(path::AbstractString)
          return replace(path, "\\\" => "/"")
        end

function parse_regex_float(txt, pattern)
  m = match(pattern, txt)
  if m === nothing
    return NaN
  end
  try
    return parse(Float64, m.captures[1])
  catch
    return NaN
  end
end

function parse_regex_int(txt, pattern)
  m = match(pattern, txt)
  if m === nothing
    return missing
  end
  try
    return parse(Int, m.captures[1])
  catch
    return missing
  end
end

function parse_regex_string(txt, pattern)
  m = match(pattern, txt)
  if m === nothing
    return ""
  end
end

```

```

        return strip(string(m.captures[1]))
    end

function executar_ampl_nlp(problema::String, solver::String)
    mod_file = joinpath(AMPL_SOURCE_DIR, problema * ".mod")
    run_file = joinpath(AMPL_RUN_DIR, problema * "_" * solver * ".run")
    log_file = joinpath(AMPL_LOG_DIR, problema * "_" * solver * ".log")

    if !AMPL_DISPONIVEL
        registrar_resultado!(
            parte = "Parte 1",
            fonte = "AMPL/NLP Benchmark",
            formato = "AMPL",
            problema = problema,
            solver = solver,
            status = "AMPL_NAO_ENCONTRADO",
            mensagem = "Executável AMPL não localizado pelo Julia. Caminho tentado: "
        )
        return
    end

    if !isfile(mod_file)
        registrar_resultado!(
            parte = "Parte 1",
            fonte = "AMPL/NLP Benchmark",
            formato = "AMPL",
            problema = problema,
            solver = solver,
            status = "ARQUIVO_NAO_ENCONTRADO",
            mensagem = mod_file
        )
        return
    end

    opt_name, opt_value = solver_options_ampl[solver]
    mod_ampl = ampl_path(mod_file)

    script = """
reset;
option solver $solver;
option $opt_name \"$opt_value\";
model \"$mod_ampl\";
solve;
printf \"AMPL_STATUS_BEGIN %s AMPL_STATUS_END\\n\\n\", solve_result;
printf \"AMPL_MESSAGE_BEGIN %s AMPL_MESSAGE_END\\n\\n\", solve_message;
printf \"AMPL_OBJECTIVE_BEGIN %.16g AMPL_OBJECTIVE_END\\n\\n\", _obj[1];
quit;
"""

    open(run_file, "w") do io
        write(io, script)
    end

    tempo = @elapsed begin
        try
            open(log_file, "w") do io
                run(pipeline(Cmd([AMPL_EXE, run_file]), stdout = io, stderr = io))
            end
        catch e
            open(log_file, "a") do io
                println(io, "JULIA_ERROR_BEGIN ", sprint(showerror, e), " JULIA_ERROR_END")
            end
        end
    end
end

```

```

saida = isfile(log_file) ? read(log_file, String) : ""

status = parse_regex_string(saida, r"AMPL_STATUS_BEGIN\s*(.*?)\s*AMPL_STATUS_END"
mensagem = parse_regex_string(saida, r"AMPL_MESSAGE_BEGIN\s*(.*?)\s*AMPL_MESSAGE_END"
objetivo = parse_regex_float(saida, r"AMPL_OBJECTIVE_BEGIN\s*([-\+]?\d*\.\d+([eE][+-]?\d+)?)")
iteracoes = parse_regex_int(saida, r"Number of Iterations\.\d+\s*(\d+)")

if isempty(status)
    status = occursin("JULIA_ERROR_BEGIN", saida) ? "ERRO_EXECUCAO" : "SEM_STATUS"
end

if isempty(mensagem)
    mensagem = texto_curto(saida)
end

registrar_resultado!(
    parte = "Parte 1",
    fonte = "AMPL/NLP Benchmark",
    formato = "AMPL",
    problema = problema,
    solver = solver,
    status = status,
    objetivo_obtido = objetivo,
    objetivo_referencia = NaN,
    tempo_segundos = tempo,
    iteracoes = iteracoes,
    mensagem = mensagem
)
end

```

Out[6]: executar_ampl_nlp (generic function with 1 method)

7. Teste rápido — AMPL/NLP

Esta célula testa um problema antes da execução completa.

In [7]: executar_ampl_nlp("qcqp500-3c", "ipopt")
last(resultados, 1)

Out[7]: 1×12 DataFrame

Row	parte	fonte	formato	problema	solver	status	objetivo_obtido	objetivo_referencia	tempo_segundos	iteracoes	mensagem
	String	String	String	String	String	String	Float64	Float64	Float64	Int64	String
1	Parte 1	AMPL/NLP Benchmark	AMPL	qcqp500-3c	ipopt	limit	46757.1	NaN	0.0	0	

In [8]: executar_ampl_nlp("c1n1beam", "ipopt")
last(resultados, 1)

Out[8]: 1×12 DataFrame

Row	parte	fonte	formato	problema	solver	status	objetivo_obtido	objetivo_referencia
	String	String	String	String	String	String	Float64	Float64
1	Parte 1	AMPL/NLP Benchmark	AMPL	cnlbeam	ipopt	limit	345.638	NaN

8. Execução completa — 47 problemas AMPL/NLP

A célula seguinte executa os 47 problemas para os solvers listados. A cada execução, os resultados parciais são salvos em arquivo `.csv`.

```
In [9]: # Execução segura da Parte 1 – 47 problemas AMPL/NLP apenas com Ipopt

for problema in problemas_ampl
    println("\nExecutando ", problema, " com Ipopt")
    executar_ampl_nlp(problema, "ipopt")
    CSV.write(joinpath(RESULTS_DIR, "resultados_parciais_ampl_nlp_ipopt.csv"), resultados)
end

CSV.write(joinpath(RESULTS_DIR, "resultados_ampl_nlp_47_ipopt.csv"), resultados)

display(resultados)
```

Executando arki0003 com Ipopt

Executando arki0009 com Ipopt

Executando bearing_400 com Ipopt

Executando camshape_6400 com Ipopt

Executando c1nlbeam com Ipopt

Executando cont5_1_1 com Ipopt

Executando cont5_2_1_1 com Ipopt

Executando cont5_2_2_1 com Ipopt

Executando cont5_2_3_1 com Ipopt

Executando cont5_2_4_1 com Ipopt

Executando corkscrw com Ipopt

Executando dirichlet120 com Ipopt

Executando dtoc1nd com Ipopt

Executando dtoc2 com Ipopt

Executando elec_400 com Ipopt

Executando ex1_160 com Ipopt

Executando ex1_320 com Ipopt

Executando ex4_2_160 com Ipopt

Executando ex4_2_320 com Ipopt

Executando ex8_2_2 com Ipopt

Executando ex8_2_3 com Ipopt

Executando gasoil_3200 com Ipopt

Executando henon120 com Ipopt

Executando lane_emden120 com Ipopt

Executando marine_1600 com Ipopt

Executando NARX_CFy com Ipopt

Executando nql180 com Ipopt

Executando optmass com Ipopt

Executando pinene_3200 com Ipopt

Executando qcqp500-3c com Ipopt

Executando qcqp500-3nc com Ipopt

Executando qcqp750-2c com Ipopt

Executando qcqp750-2nc com Ipopt

Executando qcqp1000-1nc com Ipopt

Executando qcqp1000-2c com Ipopt

Executando qcqp1000-2nc com Ipopt

Executando qcqp1500-1c com Ipopt

Executando qcqp1500-1nc com Ipopt

Executando qssp180 com Ipopt

Executando robot_1600 com Ipopt

Executando robot_a com Ipopt

Executando robot_b com Ipopt

Executando robot_c com Ipopt

Executando rocket_12800 com Ipopt

Executando steering_12800 com Ipopt

Executando svanberg com Ipopt

Executando WM_CFy com Ipopt

49×12 DataFrame

Row	parte	fonte	formato	problema	solver	status	objetivo_obtido	objeti
	String	String	String	String	String	String	Float64	Float6
1	Parte 1	AMPL/NLP Benchmark	AMPL	qcqp500-3c	ipopt	limit	46757.1	
2	Parte 1	AMPL/NLP Benchmark	AMPL	clnlbeam	ipopt	limit	345.638	
3	Parte 1	AMPL/NLP Benchmark	AMPL	arki0003	ipopt	limit	-1.17994e7	
4	Parte 1	AMPL/NLP Benchmark	AMPL	arki0009	ipopt	limit	1.99173e5	
5	Parte 1	AMPL/NLP Benchmark	AMPL	bearing_400	ipopt	solved	-0.15468	
6	Parte 1	AMPL/NLP Benchmark	AMPL	camshape_6400	ipopt	solved	4.44838	
7	Parte 1	AMPL/NLP Benchmark	AMPL	clnlbeam	ipopt	limit	345.638	
8	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_1_l	ipopt	solved	2.72097	
9	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_1_l	ipopt	solved	0.000662747	
10	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_2_l	ipopt	solved	0.000519997	
11	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_3_l	ipopt	solved	0.000520782	
12	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_4_l	ipopt	solved	0.0663714	
13	Parte 1	AMPL/NLP Benchmark	AMPL	corkscrw	ipopt	limit	39.1167	
⋮	⋮	⋮	⋮	⋮	⋮	⋮		⋮
38	Parte 1	AMPL/NLP Benchmark	AMPL	qcqp1000-2nc	ipopt	ERRO_EXECUCAO	NaN	
39	Parte 1	AMPL/NLP Benchmark	AMPL	qcqp1500-1c	ipopt	ERRO_EXECUCAO	NaN	

Row	parte	fonte	formato	problema	solver	status	objetivo_obtido	objeti
	String	String	String	String	String	String	Float64	Float6
40	Parte 1	AMPL/NLP Benchmark	AMPL	qcqp1500-1nc	ipopt	ERRO_EXECUCAO	NaN	
41	Parte 1	AMPL/NLP Benchmark	AMPL	qssp180	ipopt	ERRO_EXECUCAO	NaN	
42	Parte 1	AMPL/NLP Benchmark	AMPL	robot_1600	ipopt	ERRO_EXECUCAO	NaN	
43	Parte 1	AMPL/NLP Benchmark	AMPL	robot_a	ipopt	ERRO_EXECUCAO	NaN	

Row	parte	fonte	formato	problema	solver	status	objetivo_obtido	objeti
	String	String	String	String	String	String	Float64	Float6
44	Parte 1	AMPL/NLP Benchmark	AMPL	robot_b	ipopt	ERRO_EXECUCAO	NaN	
45	Parte 1	AMPL/NLP Benchmark	AMPL	robot_c	ipopt	ERRO_EXECUCAO	NaN	
46	Parte 1	AMPL/NLP Benchmark	AMPL	rocket_12800	ipopt	ERRO_EXECUCAO	NaN	
47	Parte 1	AMPL/NLP Benchmark	AMPL	steering_12800	ipopt	ERRO_EXECUCAO	NaN	

Row	parte	fonte	formato	problema	solver	status	objetivo_obtido	objeti
	String	String	String	String	String	String	Float64	Float6

48	Parte 1	AMPL/NLP Benchmark	AMPL	svanberg	ipopt	ERRO_EXECUCAO	NaN
----	---------	--------------------	------	----------	-------	---------------	-----

49	Parte 1	AMPL/NLP Benchmark	AMPL	WM_CfY	ipopt	ERRO_EXECUCAO	NaN
----	---------	--------------------	------	--------	-------	---------------	-----

Parte 2 — COCONUT/GAMS Benchmark

A segunda parte utiliza problemas da biblioteca COCONUT/GAMS com solução de referência. Nesta etapa, os modelos selecionados são representados em Julia/JuMP a partir do formato GAMS e comparados com o valor de referência conhecido.

9. Problemas COCONUT/GAMS selecionados

```
In [10]: problemas_coconut = DataFrame(
    numero = [1062, 1063, 1064],
    problema = ["ex4_1_1", "ex4_1_2", "ex4_1_3"],
    fonte = fill("COCONUT/GAMS Benchmark", 3),
    formato = fill("GAMS convertido para JuMP", 3),
    status_referencia = fill(2, 3),
    variaveis = fill(1, 3),
    restricoes = fill(0, 3),
    objetivo_referencia = [-7.4873123649, -663.5000966105, -443.6717047411]
)
```

```
display(problemas_coconut)
```

3×8 DataFrame

Row	numero	problema	fonte	formato	status_referencia	variaveis	restricoes	obje
	Int32	String	String	String	Int32	Int32	Int32	Floa
1	1062	ex4_1_1	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	2	1	0	
2	1063	ex4_1_2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	2	1	0	
3	1064	ex4_1_3	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	2	1	0	

10. Solvers previstos para a Parte COCONUT/GAMS

Os problemas COCONUT/GAMS selecionados são executados em JuMP. Cada solver é utilizado quando sua interface Julia/JuMP estiver disponível no ambiente.

```
In [11]: solvers_jump = ["Ipopt", "MadNLP", "NLopt", "Optim", "Uno"]

function importar_pacote(pkg::Symbol)
    try
        Core.eval(Main, Expr(:import, pkg))
        return true, ""
    catch e
        return false, sprint(showerror, e)
    end
end

function criar_modelo_jump(solver::String)
    if !isdefined(Main, :Model)
        return nothing, false, "JuMP não foi carregado. Execute a célula 2 – Pacotes"
    end

    if solver == "Ipopt"
        ok, err = importar_pacote(:Ipopt)
        if !ok
            return nothing, false, err
        end
        model = Model(Ipopt.Optimizer)
        try set_optimizer_attribute(model, "max_iter", MAX_ITER) catch end
        try set_silent(model) catch end
        return model, true, ""
    elseif solver == "MadNLP"
        ok, err = importar_pacote(:MadNLP)
        if !ok
            return nothing, false, err
        end
        model = Model(MadNLP.Optimizer)
        try set_optimizer_attribute(model, "max_iter", MAX_ITER) catch end
        try set_silent(model) catch end
        return model, true, ""
    end
end
```

```

elseif solver == "NLOpt"
    ok, err = importar_pacote(:NLOpt)
    if !ok
        return nothing, false, err
    end
    model = Model(NLOpt.Optimizer)
    try set_optimizer_attribute(model, "algorithm", :LD_MMA) catch end
    try set_optimizer_attribute(model, "maxeval", MAX_ITER) catch end
    try set_silent(model) catch end
    return model, true, ""

elseif solver == "Optim"
    return nothing, false, "Optim.jl não é usado diretamente como solver JuMP ge

elseif solver == "Uno"
    ok, err = importar_pacote(:Uno)
    if !ok
        return nothing, false, err
    end
    try
        model = Model(UnO.Optimizer)
        try set_optimizer_attribute(model, "max_iter", MAX_ITER) catch end
        try set_silent(model) catch end
        return model, true, ""
    catch e
        return nothing, false, sprint(showerror, e)
    end
end

return nothing, false, "Solver não cadastrado."
end

DataFrame(
    solver = solvers_jump,
    limite_iteracoes = fill(MAX_ITER, length(solvers_jump))
)

```

Out[11]: 5×2 DataFrame

Row	solver	limite_iteracoes
	String	Int32
1	lpopt	100
2	MadNLP	100
3	NLOpt	100
4	Optim	100
5	Uno	100

11. Modelos COCONUT/GAMS convertidos para JuMP

```

In [12]: function montar_modelo_coconut!(model, problema::String)
    if problema == "ex4_1_1"
        @variable(model, -2 <= x <= 11, start = -1.2)
        @Nlobjective(
            model,

```

```

        Min,
        -x - 3.95*x^2 + 7.1*x^3 + 0.4875*x^4 - 2.08*x^5 + x^6 + 0.1
    )
elseif problema == "ex4_1_2"
    @variable(model, 1 <= x <= 2, start = 1.091)
    @NLOjective(
        model,
        Min,
        -500*x
        + 2.5*x^2
        + 1.666666666*x^3
        + 1.25*x^4
        + x^5
        + 0.8333333*x^6
        + 0.714285714*x^7
        + 0.625*x^8
        + 0.555555555*x^9
        + x^10
        - 43.6363636*x^11
        + 0.416666666*x^12
        + 0.384615384*x^13
        + 0.357142857*x^14
        + 0.3333333*x^15
        + 0.3125*x^16
        + 0.294117647*x^17
        + 0.277777777*x^18
        + 0.263157894*x^19
        + 0.25*x^20
        + 0.238095238*x^21
        + 0.227272727*x^22
        + 0.217391304*x^23
        + 0.208333333*x^24
        + 0.2*x^25
        + 0.192307692*x^26
        + 0.185185185*x^27
        + 0.178571428*x^28
        + 0.344827586*x^29
        + 0.6666666*x^30
        - 15.48387097*x^31
        + 0.15625*x^32
        + 0.1515151*x^33
        + 0.14705882*x^34
        + 0.14285712*x^35
        + 0.138888888*x^36
        + 0.135135135*x^37
        + 0.131578947*x^38
        + 0.128205128*x^39
        + 0.125*x^40
        + 0.121951219*x^41
        + 0.119047619*x^42
        + 0.116279069*x^43
        + 0.113636363*x^44
        + 0.1111111*x^45
        + 0.108695652*x^46
        + 0.106382978*x^47
        + 0.208333333*x^48
        + 0.408163265*x^49
        + 0.8*x^50
    )
elseif problema == "ex4_1_3"
    @variable(model, 0 <= x <= 10, start = 6.3)
    @NLOjective(
        model,
        Min,

```

```

        0.000089248*x
        - 0.0218343*x^2
        + 0.998266*x^3
        - 1.6995*x^4
        + 0.2*x^5
    )
else
    error("Problema COCONUT/GAMS não cadastrado: " * problema)
end
return model
end

```

Out[12]: montar_modelo_coconut! (generic function with 1 method)

12. Execução COCONUT/GAMS

```

In [13]: function executar_coconut_jump(problema::String, solver::String, objetivo_referencia
        model, ok, msg = criar_modelo_jump(solver)

        if !ok
            registrar_resultado!(
                parte = "Parte 2",
                fonte = "COCONUT/GAMS Benchmark",
                formato = "GAMS convertido para JuMP",
                problema = problema,
                solver = solver,
                status = "SOLVER_INDISPONIVEL",
                objetivo_referencia = objetivo_referencia,
                mensagem = texto_curto(msg)
            )
            return
        end

        try
            montar_modelo_coconut!(model, problema)

            tempo = @elapsed begin
                optimize!(model)
            end

            status = string(termination_status(model))
            objetivo = has_values(model) ? objective_value(model) : NaN

            registrar_resultado!(
                parte = "Parte 2",
                fonte = "COCONUT/GAMS Benchmark",
                formato = "GAMS convertido para JuMP",
                problema = problema,
                solver = solver,
                status = status,
                objetivo_obtido = objetivo,
                objetivo_referencia = objetivo_referencia,
                tempo_segundos = tempo,
                iteracoes = missing,
                mensagem = ""
            )
        catch e
            registrar_resultado!(
                parte = "Parte 2",
                fonte = "COCONUT/GAMS Benchmark",
                formato = "GAMS convertido para JuMP",
                problema = problema,

```

```

        solver = solver,
        status = "ERRO_EXECUCAO",
        objetivo_referencia = objetivo_referencia,
        mensagem = texto_curto(sprint(showerror, e))
    )
end
end

for row in eachrow(problemas_coconut)
    for solver in solvers_jump
        println("Executando ", row.problema, " com ", solver)
        executar_coconut_jump(row.problema, solver, row.objetivo_referencia)
        CSV.write(joinpath(RESULTS_DIR, "resultados_parciais_coconut_gams.csv"), resu
    end
end

CSV.write(joinpath(RESULTS_DIR, "resultados_coconut_gams.csv"), resultados)
display(resultados)

```

```

Executando ex4_1_1 com Ipopt
Executando ex4_1_1 com MadNLP
Executando ex4_1_1 com NLOpt
Executando ex4_1_1 com Optim
Executando ex4_1_1 com Uno
Executando ex4_1_2 com Ipopt
Executando ex4_1_2 com MadNLP
Executando ex4_1_2 com NLOpt
Executando ex4_1_2 com Optim
Executando ex4_1_2 com Uno
Executando ex4_1_3 com Ipopt
Executando ex4_1_3 com MadNLP
Executando ex4_1_3 com NLOpt
Executando ex4_1_3 com Optim
Executando ex4_1_3 com Uno

```


64×12 DataFrame

Row	parte	fonte	formato	problema	solver	status	objetivo
	String	String	String	String	String	String	Float64
1	Parte 1	AMPL/NLP Benchmark	AMPL	qcqp500-3c	ipopt	limit	
2	Parte 1	AMPL/NLP Benchmark	AMPL	clnlbeam	ipopt	limit	
3	Parte 1	AMPL/NLP Benchmark	AMPL	arki0003	ipopt	limit	-1.0
4	Parte 1	AMPL/NLP Benchmark	AMPL	arki0009	ipopt	limit	1.0
5	Parte 1	AMPL/NLP Benchmark	AMPL	bearing_400	ipopt	solved	-
6	Parte 1	AMPL/NLP Benchmark	AMPL	camshape_6400	ipopt	solved	
7	Parte 1	AMPL/NLP Benchmark	AMPL	clnlbeam	ipopt	limit	
8	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_1_l	ipopt	solved	
9	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_1_l	ipopt	solved	0.000
10	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_2_l	ipopt	solved	0.000

Row	parte	fonte	formato	problema	solver	status	objetivo
	String	String	String	String	String	String	Float64
11	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_3_I	ipopt	solved	0.001
12	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_4_I	ipopt	solved	0.001
13	Parte 1	AMPL/NLP Benchmark	AMPL	corkscrw	ipopt	limit	
⋮	⋮	⋮	⋮	⋮	⋮	⋮	
53	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_1	Optim	SOLVER_INDISPONIVEL	
54	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_1	Uno	SOLVER_INDISPONIVEL	
55	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_2	Ipopt	SOLVER_INDISPONIVEL	
56	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_2	MadNLP	SOLVER_INDISPONIVEL	
57	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_2	NLopt	SOLVER_INDISPONIVEL	
58	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_2	Optim	SOLVER_INDISPONIVEL	
59	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido	ex4_1_2	Uno	SOLVER_INDISPONIVEL	

Row	parte	fonte	formato	problema	solver	status	objetivo
	String	String	String	String	String	String	Float64
para JuMP							
60	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_3	Ipopt	SOLVER_INDISPONIVEL	
61	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_3	MadNLP	SOLVER_INDISPONIVEL	
62	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_3	NLopt	SOLVER_INDISPONIVEL	
63	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_3	Optim	SOLVER_INDISPONIVEL	
64	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_3	Uno	SOLVER_INDISPONIVEL	

Consolidação dos resultados

13. Resultados completos

```
In [14]: CSV.write(joinpath(REULTS_DIR, "resultados_completos.csv"), resultados)
          display(resultados)
```

64×12 DataFrame

Row	parte	fonte	formato	problema	solver	status	objetivo
	String	String	String	String	String	String	Float64
1	Parte 1	AMPL/NLP Benchmark	AMPL	qcqp500-3c	ipopt	limit	
2	Parte 1	AMPL/NLP Benchmark	AMPL	clnlbeam	ipopt	limit	
3	Parte 1	AMPL/NLP Benchmark	AMPL	arki0003	ipopt	limit	-1.0
4	Parte 1	AMPL/NLP Benchmark	AMPL	arki0009	ipopt	limit	1.0
5	Parte 1	AMPL/NLP Benchmark	AMPL	bearing_400	ipopt	solved	-
6	Parte 1	AMPL/NLP Benchmark	AMPL	camshape_6400	ipopt	solved	
7	Parte 1	AMPL/NLP Benchmark	AMPL	clnlbeam	ipopt	limit	
8	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_1_l	ipopt	solved	
9	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_1_l	ipopt	solved	0.000
10	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_2_l	ipopt	solved	0.000

Row	parte	fonte	formato	problema	solver	status	objetivo
	String	String	String	String	String	String	Float64
11	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_3_l	ipopt	solved	0.001
12	Parte 1	AMPL/NLP Benchmark	AMPL	cont5_2_4_l	ipopt	solved	0.001
13	Parte 1	AMPL/NLP Benchmark	AMPL	corkscrw	ipopt	limit	
⋮	⋮	⋮	⋮	⋮	⋮	⋮	
53	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_1	Optim	SOLVER_INDISPONIVEL	
54	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_1	Uno	SOLVER_INDISPONIVEL	
55	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_2	Ipopt	SOLVER_INDISPONIVEL	
56	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_2	MadNLP	SOLVER_INDISPONIVEL	
57	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_2	NLopt	SOLVER_INDISPONIVEL	
58	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_2	Optim	SOLVER_INDISPONIVEL	
59	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido	ex4_1_2	Uno	SOLVER_INDISPONIVEL	

Row	parte	fonte	formato	problema	solver	status	objetivo
	String	String	String	String	String	String	Float64
para JuMP							
60	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_3	Ipopt	SOLVER_INDISPONIVEL	
61	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_3	MadNLP	SOLVER_INDISPONIVEL	
62	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_3	NLopt	SOLVER_INDISPONIVEL	
63	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_3	Optim	SOLVER_INDISPONIVEL	
64	Parte 2	COCONUT/GAMS Benchmark	GAMS convertido para JuMP	ex4_1_3	Uno	SOLVER_INDISPONIVEL	

14. Resumo por solver

```
In [16]: function media_sem_nan(v)
    vals = [x for x in v if !isnan(x)]
    return isempty(vals) ? NaN : mean(vals)
end

function contar_sucessos(statuses)
    return count(s -> begin
        t = lowercase(string(s))
        occursin("solved", t) || occursin("optimal", t) || occursin("locally", t) ||
    end, statuses)
end

resumo_solver = combine(
    groupby(resultados, [:parte, :solver]),
    :problema => length => :execucoes,
    :status => contar_sucessos => :sucessos,
    :tempo_segundos => media_sem_nan => :tempo_medio,
    :gap => media_sem_nan => :gap_medio
)

display(resumo_solver)
CSV.write(joinpath(RESULTS_DIR, "resumo_por_solver.csv"), resumo_solver)
```

6×6 DataFrame

Row	parte	solver	execucoes	sucessos	tempo_medio	gap_medio
	String	String	Int32	Int32	Float64	Float64
1	Parte 1	ipopt	49	18	726.686	NaN
2	Parte 2	Ipopt	3	0	NaN	NaN
3	Parte 2	MadNLP	3	0	NaN	NaN
4	Parte 2	NLOpt	3	0	NaN	NaN
5	Parte 2	Optim	3	0	NaN	NaN
6	Parte 2	Uno	3	0	NaN	NaN



Out[16]: "results\\resumo_por_solver.csv"

15. Resumo por problema

```
In [17]: resumo_problema = combine(
    groupby(resultados, [:parte, :problema]),
    :solver => length => :tentativas,
    :status => contar_sucessos => :sucessos,
    :objetivo_obtido => media_sem_nan => :objetivo_medio,
    :tempo_segundos => media_sem_nan => :tempo_medio,
    :gap => media_sem_nan => :gap_medio
)

display(resumo_problema)
CSV.write(joinpath(RESULTS_DIR, "resumo_por_problema.csv"), resumo_problema)
```

50×7 DataFrame

25 rows omitted

Row	parte	problema	tentativas	sucessos	objetivo_medio	tempo_medio	gap_medio
	String	String	Int32	Int32	Float64	Float64	Float64
1	Parte 1	qcqp500-3c	2	0	46757.1	512.528	NaN
2	Parte 1	clnbeam	2	0	345.638	113.583	NaN
3	Parte 1	arki0003	1	0	-1.17994e7	6.63755	NaN
4	Parte 1	arki0009	1	0	1.99173e5	40.4195	NaN
5	Parte 1	bearing_400	1	1	-0.15468	147.164	NaN
6	Parte 1	camshape_6400	1	1	4.44838	24.9337	NaN
7	Parte 1	cont5_1_l	1	1	2.72097	303.087	NaN
8	Parte 1	cont5_2_1_l	1	1	0.000662747	708.435	NaN
9	Parte 1	cont5_2_2_l	1	1	0.000519997	845.635	NaN
10	Parte 1	cont5_2_3_l	1	1	0.000520782	860.234	NaN
11	Parte 1	cont5_2_4_l	1	1	0.0663714	549.128	NaN
12	Parte 1	corkscrw	1	0	39.1167	61.4046	NaN
13	Parte 1	dirichlet120	1	1	0.0350361	13128.3	NaN
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
39	Parte 1	qssp180	1	0	NaN	0.355494	NaN
40	Parte 1	robot_1600	1	0	NaN	0.368112	NaN
41	Parte 1	robot_a	1	0	NaN	0.360398	NaN
42	Parte 1	robot_b	1	0	NaN	0.35454	NaN
43	Parte 1	robot_c	1	0	NaN	0.347136	NaN
44	Parte 1	rocket_12800	1	0	NaN	11.4447	NaN
45	Parte 1	steering_12800	1	0	NaN	0.33928	NaN
46	Parte 1	svanberg	1	0	NaN	0.344701	NaN
47	Parte 1	WM_CFy	1	0	NaN	0.328918	NaN
48	Parte 2	ex4_1_1	5	0	NaN	NaN	NaN
49	Parte 2	ex4_1_2	5	0	NaN	NaN	NaN
50	Parte 2	ex4_1_3	5	0	NaN	NaN	NaN

Out[17]: "results\\resumo_por_problema.csv"

16. Melhores resultados por problema

```
In [18]: resultados_com_objetivo = filter(row -> !isnan(row.objetivo_obtido), resultados)

if nrow(resultados_com_objetivo) > 0
  melhores = combine(groupby(resultados_com_objetivo, [:parte, :problema])) do sdf
    idx = argmin(sdf.objetivo_obtido)
    return DataFrame(
      solver = sdf.solver[idx],
      status = sdf.status[idx],
```



```
        objetivo_obtido = sdf.objetivo_obtido[idx],
        objetivo_referencia = sdf.objetivo_referencia[idx],
        gap = sdf.gap[idx],
        tempo_segundos = sdf.tempo_segundos[idx]
    )
end
display(melhores)
CSV.write(joinpath(RESULTS_DIR, "melhores_resultados_por_problema.csv"), melhores)
else
    println("Nenhum objetivo numérico registrado.")
end
```

25×8 DataFrame

Row	parte	problema	solver	status	objetivo_obtido	objetivo_referencia	gap	tempo_s
	String	String	String	String	Float64	Float64	Float64	Float64
1	Parte 1	qcqp500-3c	ipopt	limit	46757.1	NaN	NaN	
2	Parte 1	clnlbeam	ipopt	limit	345.638	NaN	NaN	
3	Parte 1	arki0003	ipopt	limit	-1.17994e7	NaN	NaN	
4	Parte 1	arki0009	ipopt	limit	1.99173e5	NaN	NaN	
5	Parte 1	bearing_400	ipopt	solved	-0.15468	NaN	NaN	
6	Parte 1	camshape_6400	ipopt	solved	4.44838	NaN	NaN	
7	Parte 1	cont5_1_l	ipopt	solved	2.72097	NaN	NaN	
8	Parte 1	cont5_2_1_l	ipopt	solved	0.000662747	NaN	NaN	
9	Parte 1	cont5_2_2_l	ipopt	solved	0.000519997	NaN	NaN	
10	Parte 1	cont5_2_3_l	ipopt	solved	0.000520782	NaN	NaN	
11	Parte 1	cont5_2_4_l	ipopt	solved	0.0663714	NaN	NaN	
12	Parte 1	corkscrw	ipopt	limit	39.1167	NaN	NaN	
13	Parte 1	dirichlet120	ipopt	solved	0.0350361	NaN	NaN	
14	Parte 1	dtoc1nd	ipopt	solved	136.456	NaN	NaN	
15	Parte 1	dtoc2	ipopt	solved	2.8633	NaN	NaN	
16	Parte 1	elec_400	ipopt	limit	75583.2	NaN	NaN	
17	Parte 1	ex1_160	ipopt	solved	0.0638967	NaN	NaN	
18	Parte 1	ex1_320	ipopt	solved	0.0654351	NaN	NaN	
19	Parte 1	ex4_2_160	ipopt	solved	3.64612	NaN	NaN	
20	Parte 1	ex4_2_320	ipopt	solved	3.63917	NaN	NaN	
21	Parte 1	ex8_2_2	ipopt	solved	-552.666	NaN	NaN	
22	Parte 1	ex8_2_3	ipopt	solved	-3731.08	NaN	NaN	
23	Parte 1	gasoil_3200	ipopt	solved	0.0052366	NaN	NaN	

Row	parte	problema	solver	status	objetivo_obtido	objetivo_referencia	gap	tempo_s
	String	String	String	String	Float64	Float64	Float64	Float64
24	Parte 1	henon120	ipopt	limit	1084.59	NaN	NaN	
25	Parte 1	lane_emden120	ipopt	solved	9.34025	NaN	NaN	

```
Out[10]: "results\\melhores_resultados_por_problema.csv"
```

17. Arquivos gerados

Os arquivos de resultado são salvos na pasta `results` :

- `resultados_completos.csv`
- `resultados_ampl_nlp_47_solvers.csv`
- `resultados_coconut_gams.csv`
- `resumo_por_solver.csv`
- `resumo_por_problema.csv`
- `melhores_resultados_por_problema.csv`

```
In [ ]: readdir(RESULTS_DIR)
```